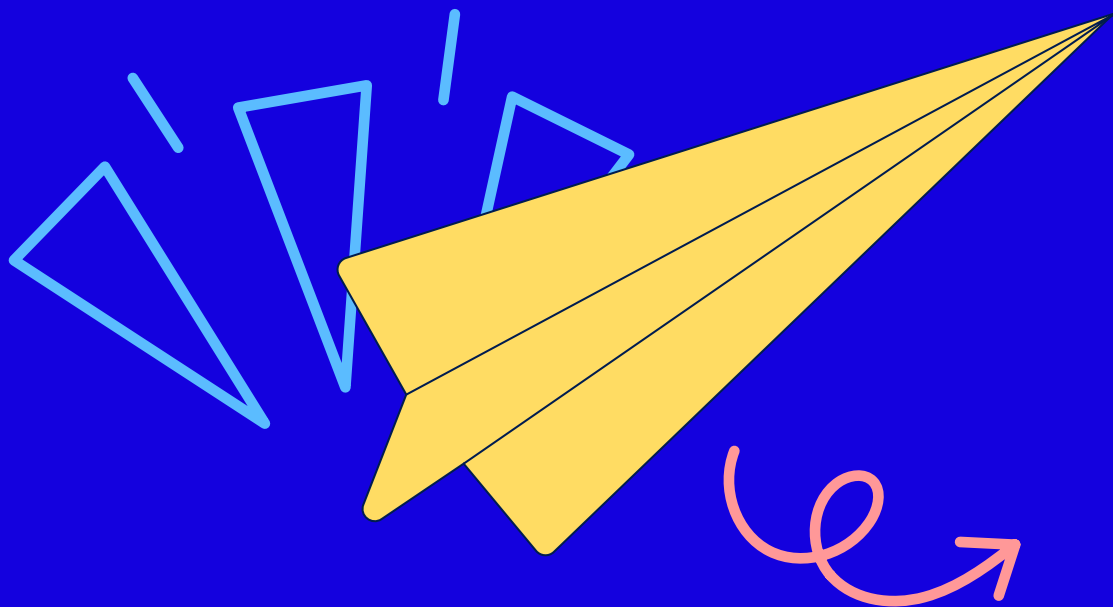
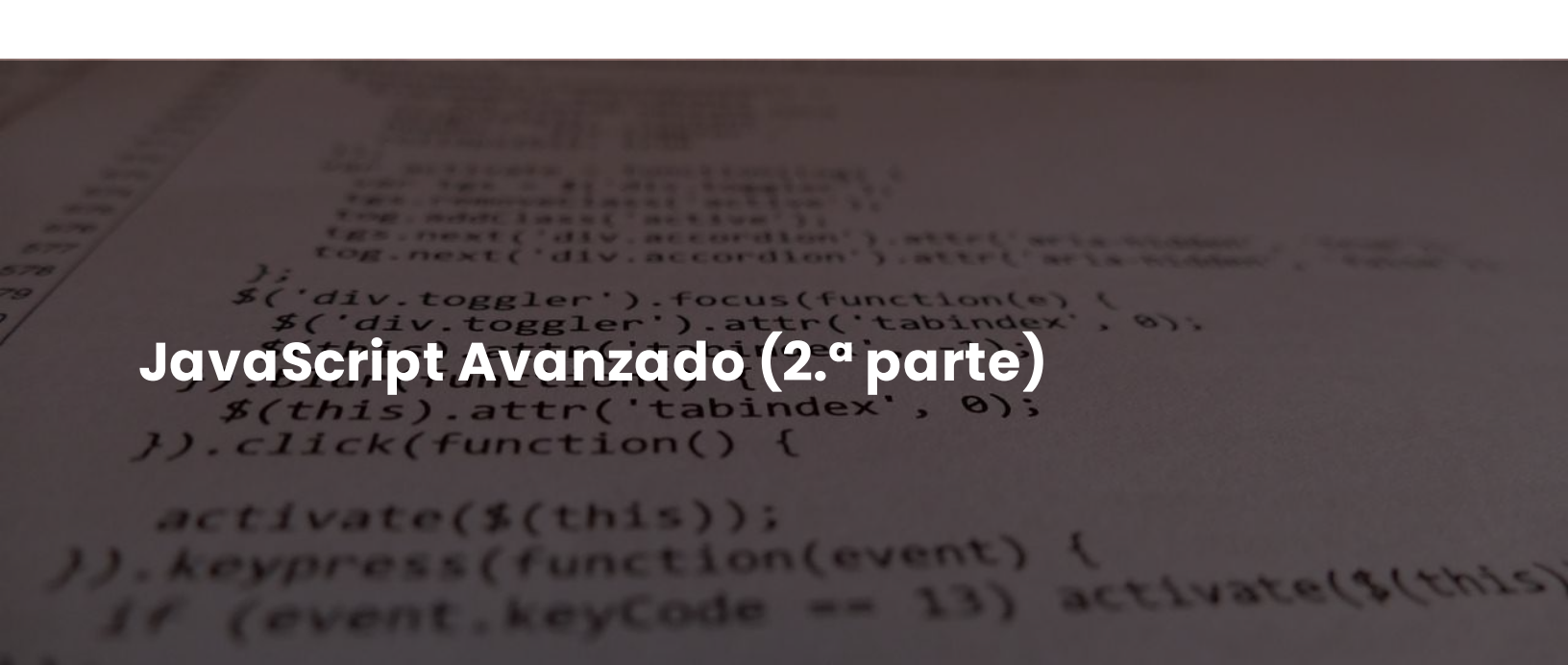




JavaScript Avanzado (2.ª parte)

Frontend



JavaScript Avanzado (2.ª parte)

Adentrarse en el universo de JavaScript puede parecer, en ocasiones, como embarcarse en una odisea intergaláctica. Hay infinidad de rincones por descubrir, desde los misteriosos **prototipos y la herencia prototipal** hasta el enigmático objeto *Object*. ¿Te has preguntado alguna vez cómo todo esto se entrelaza para crear la magia detrás de tus aplicaciones favoritas?

A lo largo de estas páginas, te llevaremos de la mano a través de estos conceptos avanzados, pero de una manera amigable y entendible. No solo nos sumergiremos en las **funciones constructoras y las clases**, que son como los ladrillos y el cemento de nuestras aplicaciones, sino que también exploraremos cómo mantener nuestras creaciones ágiles y eficientes, abordando temas como la **memoria y la optimización del rendimiento**.

Y porque sabemos que el mundo de JavaScript es vasto y en constante evolución, también te introduciremos a herramientas esenciales como **npm, transpiladores** y la fascinante **biblioteca RxJS**.

No te preocupes si todo esto suena complicado ahora; nuestro objetivo es hacer que estos temas sean comprensibles y, sobre todo, útiles para ti. Así que, ya seas un entusiasta del código o un profesional experimentado, este fastbook está diseñado pensando en ti, para que puedas navegar por el cosmos de JavaScript con confianza y curiosidad.

Autor: Robert Da Corte



Prototipos y herencia prototipal



Memoria y optimización de rendimiento



Npm



Transpiladores



RxJS



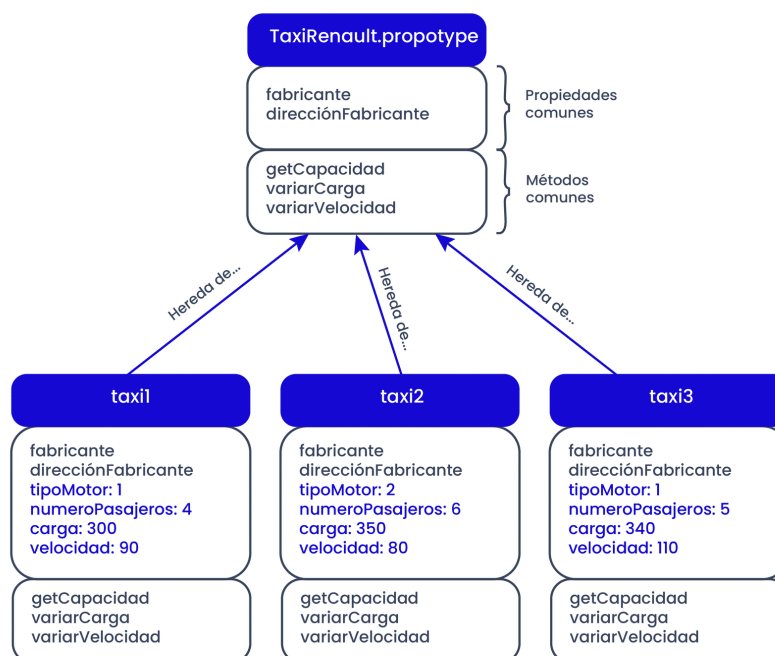
¿Qué hemos aprendido?

Prototipos y herencia prototipal

A diferencia de otros lenguajes que utilizan la herencia basada en clases, **JavaScript utiliza la herencia basada en prototipos**. Esto significa que los objetos pueden heredar propiedades y métodos directamente de otros objetos.

Prototipos

En JavaScript, casi todo es un objeto, y estos objetos están interconectados a través de una cadena de prototipos. Esta cadena es una serie de referencias a otros objetos que permiten la herencia de propiedades y métodos.



Cada objeto en JavaScript tiene una propiedad interna (a menudo referida como **[[Prototype]]**) que es una referencia a otro objeto. Este objeto referenciado es el 'prototipo' del objeto original. La propiedad **[[Prototype]]** no es directamente accesible en la mayoría de los entornos; pero, en navegadores y Node.js, puedes acceder a ella usando la propiedad **__proto__** (aunque su uso no es recomendado en producción debido a problemas de compatibilidad y rendimiento).

Cuando intentas acceder a una propiedad o método en un objeto y este no se encuentra en el objeto mismo, JavaScript buscará en el prototipo del objeto (es decir, en el objeto referenciado por **[[Prototype]]**). Si no lo encuentra allí, continuará buscando en el prototipo del prototipo, y así sucesivamente, hasta que llegue al final de la cadena de prototipos o encuentre la propiedad/método deseado.



Recuerda: si llega al final de la cadena de prototipos y no ha encontrado la propiedad o método, devolverá *undefined*.

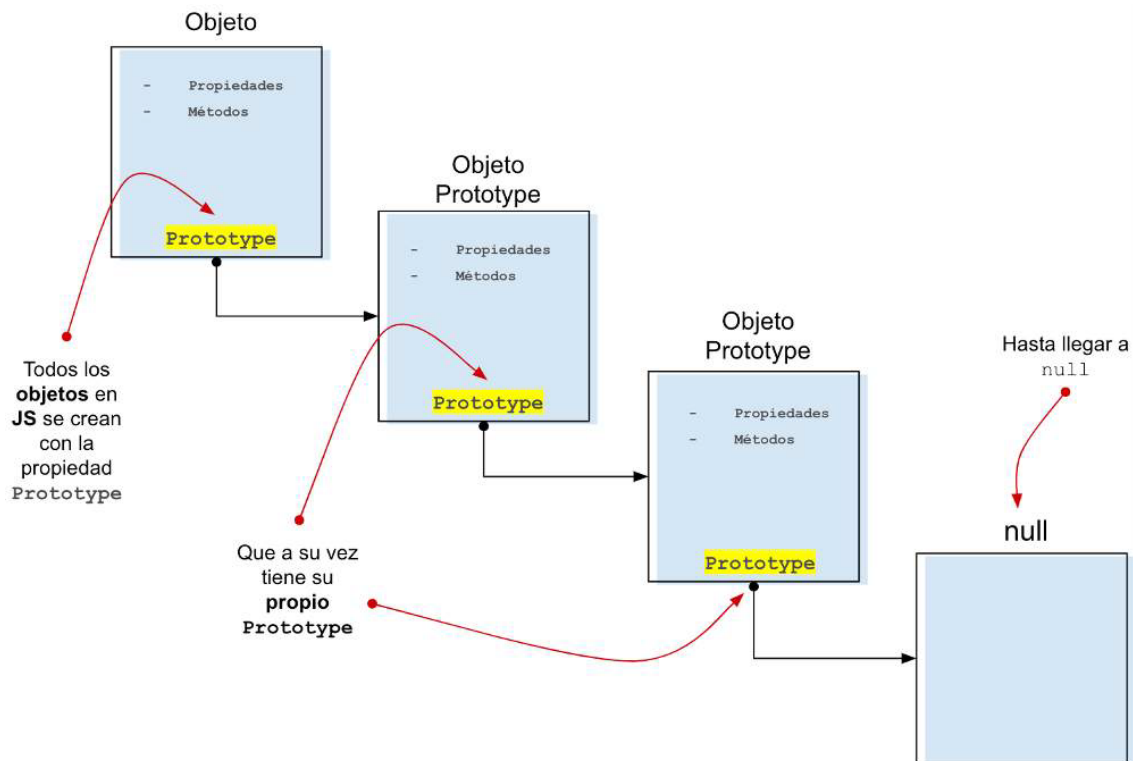
Object

Es el punto de partida para todos los objetos. Es el objeto base del cual todos los demás objetos heredan propiedades y métodos. El constructor de **Object** crea un contenedor de objeto para un valor dado. Si se le pasa un valor, devolverá un objeto de un tipo que corresponda al valor dado. Si no se le pasa un valor, devolverá un objeto vacío.

```
// Object

const obj1 = new Object();           // Crea un objeto vacío
const obj2 = new Object(undefined); // Crea un objeto vacío
const obj3 = new Object(null);       // Crea un objeto vacío
const obj4 = new Object("texto");    // Crea un objeto String
```

Todos los objetos en JavaScript heredan, de una forma u otra, del objeto **Object.prototype**. Este es el final de la cadena de prototipos para la mayoría de los objetos. Es decir, si sigues la cadena de prototipos de cualquier objeto, eventualmente llegarás a **Object.prototype**.



Dado que todos los objetos heredan de **Object**, todos los objetos heredan las propiedades y métodos de *Object.prototype*. Algunos de estos incluyen:

- ***Object.prototype.constructor***: hace referencia al constructor del objeto.
- ***Object.prototype.hasOwnProperty(prop)***: devuelve un booleano indicando si el objeto tiene la propiedad especificada como propiedad propia (no heredada).
- ***Object.prototype.isPrototypeOf(obj)***: devuelve un booleano indicando si el objeto especificado está en la cadena de prototipos del objeto.
- ***Object.prototype.toString()***: devuelve una representación de cadena del objeto.
- ***Object.prototype.valueOf()***: devuelve el valor primitivo del objeto especificado.



¡Recomendación! Para profundizar más en métodos y propiedades del objeto *Object*, visita la página oficial de [Mozilla](#).

Funciones constructoras

Una función constructora es una función que **crea un objeto**. Las funciones constructoras, por convención, comienzan con una letra mayúscula para diferenciarlas de las funciones normales.

Una función constructora es esencialmente una plantilla para crear *objetos*. Define propiedades y métodos que estarán presentes en el *objeto* que se creará.

```
// Funcion constructora

function Car(brand, model) {
  this.brand = brand;
  this.model = model;
}
```

En el ejemplo anterior, **Car** es una función constructora que acepta dos parámetros: **brand** y **model**.

Ahora procederemos a crear dos coches:

```
// Funcion constructora

function Car(brand, model) {
  this.brand = brand;
  this.model = model;
}

const car1 = new Car('Toyota', 'Corolla');
console.log(car1.brand); // Toyota
console.log(car1.model); // Corolla

const car2 = new Car('Ford', 'Explorer');
console.log(car2.brand); // Ford
console.log(car2.model); // Explorer
```

Al crear estos coches, lo que hemos hecho son **instancias** de objetos de la *plantilla* **Car** donde ambos poseen dos propiedades (**model y brand**), pero cada uno de ellos con valores diferentes, es decir, cada instancia posee valores únicos propios. Como hemos visto, hemos invocado una función constructora con la palabra clave **new**, y sucede lo siguiente:

- **Creación de un nuevo objeto:** se crea un nuevo objeto vacío.
- **Establecimiento del prototipo:** el prototipo del nuevo objeto (su propiedad **[[Prototype]]**) se establece en el objeto referenciado por la propiedad *prototype* de la función constructora.
- **Vinculación de *this*:** la palabra clave **this** dentro de la función constructora hace referencia al nuevo objeto creado.
- **Ejecución del código de la función constructora:** se ejecuta el cuerpo de la función constructora, inicializando las propiedades del nuevo objeto.
- **Devolución del nuevo objeto:** si la función constructora no devuelve explícitamente un objeto, se devuelve el nuevo objeto creado.

Debido a que los nuevos objetos **car1** y **car2** apuntan a la función *Car*, podríamos implementar una función al **prototype** de **Car** y, de esa manera, ambas instancias tendrán acceso dicha función.

```
// Función constructora

function Car(brand, model) {
  this.brand = brand;
  this.model = model;
}

const car1 = new Car('Toyota', 'Corolla');
console.log(car1.brand); // Toyota
console.log(car1.model); // Corolla

const car2 = new Car('Ford', 'Explorer');
console.log(car2.brand); // Ford
console.log(car2.model); // Explorer

Car.prototype.startEngine = function() {
  console.log(`${this.brand} ${this.model} está arrancando el motor`);
}

car1.startEngine(); // Toyota Corolla está arrancando el motor
car2.startEngine(); // Ford Explorer está arrancando el motor
```

Class

La palabra reservada *class* en JavaScript fue introducida en ES6 y proporciona una sintaxis más clara y concisa para crear objetos y lidiar con la herencia. La *class* en JavaScript es **sugar syntax** lo que significa es que, detrás de escena, todavía utilizan la herencia prototipal.

Para declarar una clase, simplemente usas la palabra clave *class* seguida del nombre de la clase:

```
// Class

class Car {
  constructor(model, year) {
    this.model = model;
    this.year = year;
  }

  startEngine() {
    console.log(`El ${this.model} de ${this.year} está arrancando el motor.`);
  }

  drive() {
    console.log(`El ${this.model} está en marcha.`);
  }
}
```

Como vemos en el ejemplo, al declarar la clase nos encontramos con una función especial llamada **constructor**. El método **constructor** es un método especial que se utiliza para crear e inicializar un objeto creado a partir de una clase. Se ejecuta automáticamente cuando se crea una nueva instancia de la clase.

Los métodos que se definen dentro de la clase (fuera del constructor) son métodos de instancia, lo que significa que están disponibles en todas las instancias de la clase.

```
// Class

class Car {
  constructor(model, year) {
    this.model = model;
    this.year = year;
  }

  startEngine() {
    console.log(`El ${this.model} de ${this.year} está arrancando el motor.`);
  }

  drive() {
    console.log(`El ${this.model} está en marcha.`);
  }
}

const car1 = new Car("Corolla", "2023")
car1.startEngine() // El Corolla de 2023 está arrancando el motor.
```

La forma en la que heredamos con clases también cambia, lo hacemos mediante el uso de las palabras clave:

- **extends:** se utiliza para crear una clase como hija de otra clase.
- **super:** se utiliza para llamar al constructor de la clase padre.

```
// Class: Herencia

class Toyota extends Car {
  constructor(model, year, hybrid) {
    super(model, year);
    this.hybrid = hybrid; // Indica si el coche es híbrido o no
  }

  ecoMode() {
    if (this.hybrid) {
      console.log(`El ${this.model} está en modo ecológico.`);
    } else {
      console.log(`El ${this.model} no es un híbrido.`);
    }
  }
}
```

En el ejemplo anterior vemos cómo la clase **Toyota extiende** de **Car** y puede tener propiedades y métodos específicos para los coches Toyota. A través del método **super** llama al **constructor** de la clase **Car** para inicializar las propiedades que hereda de esta.

Veamos otro ejemplo, ahora con una clase llamado **Ford**:

```
// Class: Herencia

class Ford extends Car {
  constructor(model, year, truck) {
    super(model, year);
    this.truck = truck; // Indica si el coche es una camioneta o no
  }

  towMode() {
    if (this.truck) {
      console.log(`El ${this.model} está en modo de remolque.`);
    } else {
      console.log(`El ${this.model} no es una camioneta.`);
    }
  }
}
```

Comparando ambos ejemplos podemos ver el poder de la herencia, ambas clases heredan propiedades y métodos base de la clase **Car**, pero también pueden tener propiedades y métodos únicos. Esto lo podemos ver mejor en el siguiente ejemplo:

```
// Class: Herencia

const camry = new Toyota("Camry", 2023, true);
camry.startEngine(); // El Camry de 2023 está arrancando el motor.
camry.ecoMode();     // El Camry está en modo ecológico.

const f150 = new Ford("F-150", 2023, true);
f150.startEngine();  // El F-150 de 2023 está arrancando el motor.
f150.towMode();      // El F-150 está en modo de remolque.
```

Memoria y optimización de rendimiento

La gestión eficiente de la memoria es crucial para garantizar que las aplicaciones JavaScript se ejecuten de manera rápida y confiable. Aunque JavaScript es un lenguaje con recolección de basura, aún es posible encontrar problemas de rendimiento y fugas de memoria si no se es cuidadoso.

JavaScript utiliza un **garbage collector** (recolector de basura), lo que significa que automáticamente libera la memoria que ya no está en uso. Lo hace mediante un proceso llamado "*mark-and-sweep*". El motor de JavaScript marca las variables que están en uso y luego 'barre' las que no están marcadas, liberando su memoria.

Si bien la mayoría de las veces el *garbage collector* se hará cargo de liberar memoria, hay ocasiones en las que no servirá del todo debido a las malas prácticas en nuestro código, lo que ocasionará un *memory leak* (fuga de memoria).

Las *memory leaks* en JavaScript ocurren cuando los objetos ya innecesarios no son liberados por el *garbage collector*, porque todavía hay referencias a ellos en alguna parte del código.

A lo largo del tiempo, estas fugas pueden acumularse y consumir una cantidad significativa de memoria, lo que puede llevar a una degradación del rendimiento o incluso a que la aplicación se bloquee.

Los siguientes puntos nos sirven para **identificar y prevenir** *memory leaks*:

- **Variables globales:** evita las variables globales innecesarias. Las variables que nunca se liberan pueden causar memory leaks.
- **Event listeners:** si agregas *listeners* de eventos a un elemento DOM, asegúrate de eliminarlos cuando ya no sean necesarios.
- **Referencias circulares:** en el pasado, las referencias circulares (dos objetos que se referencian mutuamente) eran una causa común de memory leaks, especialmente cuando se interactuaba con el DOM. Los navegadores modernos han mejorado en este aspecto, pero sigue siendo algo para tener en cuenta.
- **Closures:** aunque son una característica poderosa de JavaScript, si no se usan correctamente, pueden causar memory leaks. Asegúrate de entender cómo funcionan y cuándo mantener o liberar referencias.
- **Timers y Intervals:** si usas *setTimeout* o *setInterval*, asegúrate de limpiarlos con *clearTimeout* o *clearInterval* cuando ya no sean necesarios.

Existen herramientas que nos permiten **monitorizar y depurar** nuestro código para identificar posibles problemas, entre ellas destacamos las siguientes:

- **DevTools de Chrome:** una serie de herramientas para perfilado y depuración. La pestaña '*Memory*' te permite tomar instantáneas de la memoria y compararlas, lo que es útil para identificar fugas de memoria.

- **Node.js y --inspect:** si estás trabajando con Node.js, puedes usar el *flag* `--inspect` para habilitar la depuración y, luego, conectar las *DevTools* de Chrome para analizar el rendimiento y la memoria.
- **Lighthouse:** herramienta automatizada que puede ayudarte a identificar problemas de rendimiento en aplicaciones web.

Utilizar herramientas de perfilado y depuración y tener un conocimiento sólido de cómo funciona la memoria en JavaScript es esencial para construir aplicaciones eficientes y de alto rendimiento.

Npm

Node.js, originalmente diseñado para ejecutar JavaScript en el lado del servidor, ha tenido un impacto significativo en el desarrollo *frontend*. Aunque Node.js en sí mismo no se ejecuta en el navegador, ha impulsado una serie de herramientas y tecnologías que han revolucionado el flujo de trabajo *frontend*.

Node incorpora en sí un gestor de paquetes de JavaScript llamado **npm**.

Un gestor de paquetes es una herramienta que facilita la gestión (instalación, actualización, configuración y eliminación) de paquetes de software en un sistema operativo o en un entorno de desarrollo.

Un paquete, en este contexto, es una colección de archivos que conforman una aplicación, una biblioteca o un módulo de software. En el caso de npm es una herramienta de Node.js que permite a los desarrolladores instalar, compartir y gestionar paquetes de código. Para los desarrolladores frontend, esto significa fácil acceso a:

- **Bibliotecas** (como React, Vue, Lodash).
- **Herramientas** (como Babel, Webpack, ESLint).
- **Componentes y estilos reutilizables.**

Npm actúa como un repositorio en línea para publicar y compartir paquetes de código. Estos paquetes pueden ser bibliotecas, herramientas, frameworks o cualquier fragmento de código que los desarrolladores deseen compartir. Cada paquete en npm puede tener múltiples versiones, lo que permite a los desarrolladores especificar qué versión de un paquete quieren utilizar.

Para poder utilizar npm en nuestro proyecto, necesitamos inicializarlo con el siguiente comando en una terminal:



Esto iniciará un asistente que te hará varias preguntas sobre tu proyecto, como el nombre, la versión, la descripción, el punto de entrada, etc. Si deseas saltarte el asistente y generar un *package.json* con valores predeterminados, puedes usar:



```
npm init -y
```

Al finalizar, se creará un archivo **package.json** en tu directorio con la configuración que hayas especificado.



El **package.json** es un archivo fundamental en proyectos que utilizan *Node.js* y *npm*. Define las propiedades y configuraciones de un proyecto, incluidas sus dependencias, scripts, información del autor y más.

Una vez que tengas tu *package.json*, puedes comenzar a agregar dependencias. Aquí te muestro cómo hacerlo:



```
npm install lodash
```

Esto instalará *lodash* y agregará la entrada correspondiente en la sección 'dependencies' de tu *package.json*.

Recuerda: puedes verificarlas abriendo tu archivo *package.json*. Deberías ver las dependencias y sus versiones en la sección *dependencies*.

Además, notarás que se ha creado una nueva carpeta llamada ***node_modules*** en tu directorio. Esta carpeta contiene todos los paquetes que has instalado junto con sus propias dependencias.

A partir de aquí, puedes continuar desarrollando tu proyecto, agregar más dependencias según sea necesario y aprovechar las muchas herramientas y paquetes disponibles en el ecosistema *npm*.

Transpiladores

Con la rápida evolución de los lenguajes de programación y las plataformas, a menudo surge la necesidad de adoptar nuevas características o sintaxis antes de que estén ampliamente soportadas. Los transpiladores permiten a los desarrolladores escribir código utilizando características modernas y, luego, convertir ese código a una versión que sea compatible con plataformas o entornos más antiguos.

Un transpilador (o compilador de origen a origen) toma el código fuente escrito en un lenguaje o sintaxis y lo convierte a otro lenguaje o sintaxis. En el contexto de JavaScript, esto a menudo se refiere a convertir características modernas de ES6/ES7+ a una versión más antigua de JavaScript (como ES5), que es ampliamente compatible con la mayoría de los navegadores. Esto permite que tu código funcione en una amplia gama de navegadores, incluso si no admiten las últimas características de JavaScript.

Babel es el transpilador más popular en el ecosistema de JavaScript. Con Babel, los desarrolladores pueden escribir código utilizando la última sintaxis de JavaScript, y Babel se encargará de convertirlo en código que sea compatible con versiones anteriores de JavaScript.

Para **configurar Babel** necesitamos instalar las siguientes dependencias con npm:

A terminal window with a dark background and three colored window control buttons (red, yellow, green) in the top left corner. The command 'npm install --save-dev @babel/core @babel/cli @babel/preset-env' is written in a light-colored monospace font.

```
npm install --save-dev @babel/core @babel/cli @babel/preset-env
```

Estos paquetes son los siguientes:

- **@babel/core**: el núcleo de Babel.
- **@babel/cli**: herramientas de línea de comandos para Babel.
- **@babel/preset-env**: un *preset* que permite transformar el código JavaScript moderno a una versión que sea compatible con la mayoría de los navegadores actuales.

A continuación, crea un fichero llamado **.babelrc** en la raíz de tu proyecto. Este archivo contendrá la configuración de Babel. Añade lo siguiente:

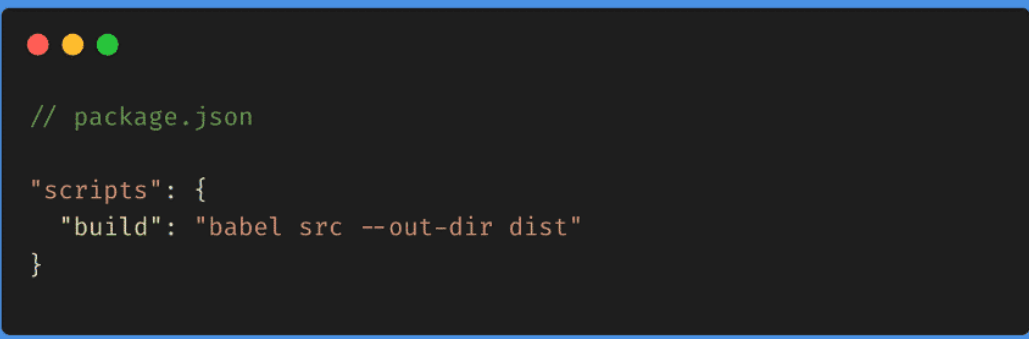


```
// .babelrc

{
  "presets": ["@babel/preset-env"]
}
```

El preset **@babel/preset-env** transformará automáticamente tu código JavaScript utilizando las transformaciones y [polyfills](#) adecuados según el entorno de destino (por defecto, transforma para una amplia gama de navegadores).

En tu *package.json*, puedes agregar un script para transpilar tu código. Suponiendo que tu código fuente esté en una carpeta llamada *src* y quieras que el código transpilado vaya a una carpeta llamada *dist*, puedes hacer lo siguiente:



```
// package.json

"scripts": {
  "build": "babel src --out-dir dist"
}
```

Con el *script* que creaste anteriormente podrás transpilar tu código ejecutando lo siguiente:



Una vez configurado Babel, puedes escribir tu código utilizando las características más recientes de JavaScript y estar seguro de que será transformado a una versión que sea compatible con la mayoría de los navegadores. Esta configuración es fundamental para muchos proyectos *frontend* modernos y facilita el desarrollo utilizando las últimas y mejores características del lenguaje.

RxJS

RxJS es una biblioteca para programación reactiva utilizando observables, lo que facilita la composición de código asíncrono o basado en eventos.

RxJS es ampliamente utilizado en el desarrollo *frontend*, especialmente en combinación con *frameworks* y bibliotecas como **Angular**, pero también es aplicable en otros contextos de JavaScript.

Entre las **funcionalidades** más usadas de RxJS tenemos las siguientes:

- **Observables:** representan una colección de valores futuros o eventos. Puedes pensar en un observable como una versión más avanzada de una promesa que puede emitir múltiples valores con el tiempo en lugar de un solo valor.
- **Observer:** conjunto de *callbacks* que sabe cómo escuchar los valores entregados por el observable. Estos callbacks suelen ser: *next*, *error* y *complete*.
- **Subscription:** es lo que se obtiene cuando se *suscribe* un *observer* a un observable. Representa la ejecución en curso de un observable.

- **Operators:** funciones puras que permiten la composición funcional de observables, transformando, filtrando, combinando y creando observables. Algunos operadores comunes son **map**, **filter**, **merge**, y **concat**.
- **Subject:** especie de puente o proxy que actúa tanto como un observable como un *observer*. Es un emisor de eventos múltiples y es lo que se utiliza para hacer *multicast* de un valor o evento a múltiples observadores.
- **Schedulers:** controladores centralizados para permitir la concurrencia, permitiendo controlar cuándo se inicia la ejecución y cuándo se procesan las notificaciones.

amos a desglosar un **ejemplo** que utiliza todos estos conceptos de RxJS: observables, bserver, subscription, operators, subject y schedulers.

ara comenzar, necesitarás tener RxJS instalado en tu proyecto.



continuación, crearemos un **observable** simple que emite números del 1 al 5 con un intervalo de tiempo.

```
// Conceptos Rxjs: Observable

import { interval } from 'rxjs'; // Emite una secuencia de numeros a lo largo del tiempo que se le pasa
import { take } from 'rxjs/operators'; // Operators

const simpleObservable = interval(1000).pipe(take(5));
```

Después, definiremos un **observer** que es un objeto con callbacks para manejar los valores emitidos, errores y la notificación de completado.

```
// .babelrc

{
  "presets": ["@babel/preset-env"]
}
```

Ahora, nos **suscribimos** al observable usando el **observer**.

```
// Conceptos Rxjs: Subscription

const subscription = simpleObservable.subscribe(simpleObserver); // Se crea una suscripción
```

Continuando ahora con los **operators**, vamos a transformar los valores emitidos multiplicándolos por 2.

```
// Conceptos Rxjs: Operators

import { map } from 'rxjs/operators';

const transformedObservable = simpleObservable.pipe(
  map(value => value * 2)
);

// Suscripción
const transformedSubscription = transformedObservable.subscribe(simpleObserver);
```

Ahora usaremos un **subject**, recordemos que es como un observable, pero puede emitir valores a múltiples observers. Es *multicast*, mientras que un observable regular es *unicast*.

```
// Conceptos Rxjs: Subject

import { Subject } from 'rxjs';

const simpleSubject = new Subject();

// Suscripción 1
simpleSubject.subscribe(simpleObserver);

// Suscripción 2
simpleSubject.subscribe({
  next: value => console.log(`Desde 2do observer: ${value}`)
});

// Emitiendo valores
simpleSubject.next('Hola');
simpleSubject.next('Mundo');
```

Y por último tenemos los **Schedulers** que controlan cuándo se inicia una suscripción y cuándo se publican las notificaciones. Vamos a usar el **asyncScheduler** para retrasar la ejecución de nuestro observable.

```
// Conceptos Rxjs: Schedulers

import { asyncScheduler } from 'rxjs';
import { observeOn } from 'rxjs/operators';

const delayedObservable = simpleObservable.pipe(
  observeOn(asyncScheduler, 2000) // Retraso de 2 segundos
);

// Suscripción
delayedObservable.subscribe(simpleObserver);
```

Para concluir es importante recordar **que debemos cerrar las suscripciones que tengamos**, ya que de no hacerlo tendríamos *memory leaks*.

```
// Conceptos Rxjs

subscription.unsubscribe();
```

Este ejemplo te da una visión general de cómo trabajar con los conceptos fundamentales de RxJS. En la práctica, la combinación de estos conceptos te permite manejar flujos de datos asíncronos de manera declarativa, lo que puede simplificar significativamente el código en aplicaciones complejas.

 **¡Importante! Para profundizar más en la librería, visita la documentación oficial de [Rxjs](#).**

¿Qué hemos aprendido?

Este viaje nos ha dotado de un arsenal de conocimientos y habilidades, preparándonos para navegar con confianza y destreza en el vasto mar del desarrollo en JavaScript:



Hemos explorado la versatilidad y profundidad del objeto **'Object'**, que actúa como la columna vertebral de muchos de nuestros desarrollos.



Las **funciones constructoras y las clases**, por otro lado, nos han enseñado las técnicas para modelar y construir componentes robustos y cohesivos en nuestras aplicaciones.



Paralelamente, nuestros estudios sobre **memoria y optimización de rendimiento** nos han equipado con las herramientas esenciales para garantizar que nuestras creaciones sean rápidas, eficientes y listas para enfrentar las demandas del mundo real.



El mundo de **npm** nos abrió las puertas a un vasto ecosistema de herramientas y bibliotecas. Esta inmersión nos permitió aprovechar el trabajo colectivo de la comunidad, potenciando nuestros proyectos y ampliando nuestras posibilidades.



Con los **transpiladores**, hemos aprendido la magia detrás de la transformación del código, permitiéndonos escribir de manera moderna y avanzada, pero manteniendo la compatibilidad y funcionalidad en diversos entornos.



RxJS, con su enfoque en la programación reactiva, nos ha ofrecido una nueva perspectiva sobre el flujo de datos y la interacción, enriqueciendo nuestra capacidad para desarrollar aplicaciones dinámicas y reactivas.

Sigue aprendiendo y profundizando en tus conocimientos, te animo a explorar en la amplia documentación y recursos disponibles.

¡Enhorabuena! Fastbook superado